OOP and Polymorphism with C



Learning Session by Dieter Niklaus

2025-12-17



Agenda

- Motivation
- The three main OOP concepts
- OOP with C
- Usage Example
- Conclusion

Motivation

- Structure large projects
- Reusability & maintainability
- enable OOP with C
- use GoF Design Patterns with C

The three main OOP concepts

Encapsulation

Inheritance

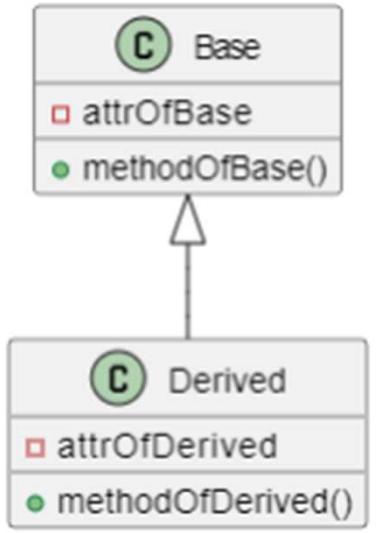
Polymorphism

Encapsulation

- Combine data and the functions working on that data
- Interface for access control, ensure controlled access to member values and functions
- Reveal only information about the "what" (functionality) of a class, but not the "how" (the internal representation)
- Defines an interface to the outside and document it at the same time
- Provide significantly improved testability, stability, changeability, and maintainability of the software or its modules

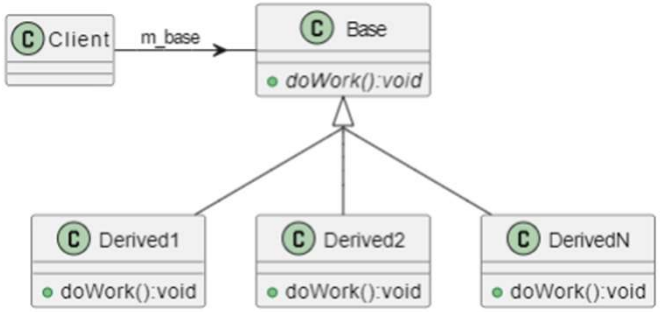
Inheritance

- Derive new "classes" from existing ones
- Add new functionality to the base class by the implementation of the derived class



Polymorphism

- Uniform interface, different implementations
- Builds directly on inheritance
  - Identifier that can take on obj. of different data types is a base class type
  - and the obj. of different datatypes are all derived from the base class type



OOP with C – Overview

- struct as class
- Function pointers for methods
- Virtual function table concept for polymorphism

Encapsulation in C

- struct + function pointers

```
typedef struct Base Base;
typedef struct BaseAttr BaseAttr;

struct BaseAttr
{
    char* name;
}

struct Base
{
    BaseAttr attr;
    const char* (*getName)(Base const* const me);
} ;
```

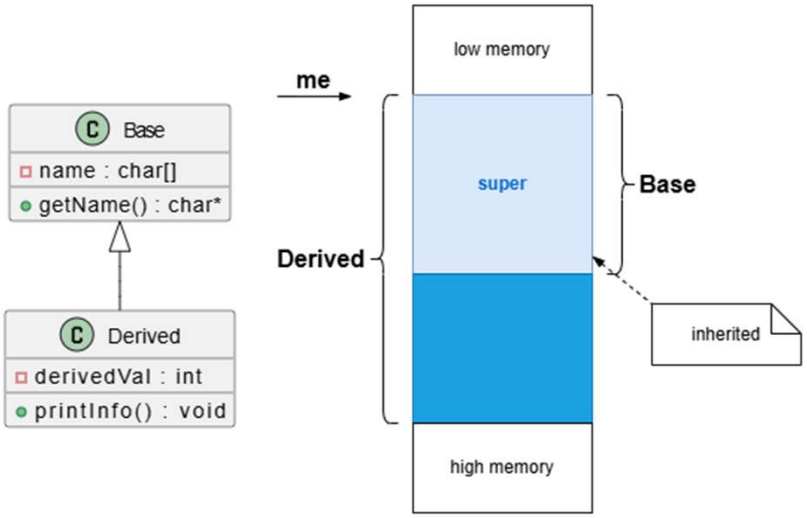
Inheritance in C

- struct embedding - base class as first member

```
typedef struct Derived Derived;
typedef struct DerivedAttr DerivedAttr;

struct DerivedAttr
{
    int derivedVal;
}

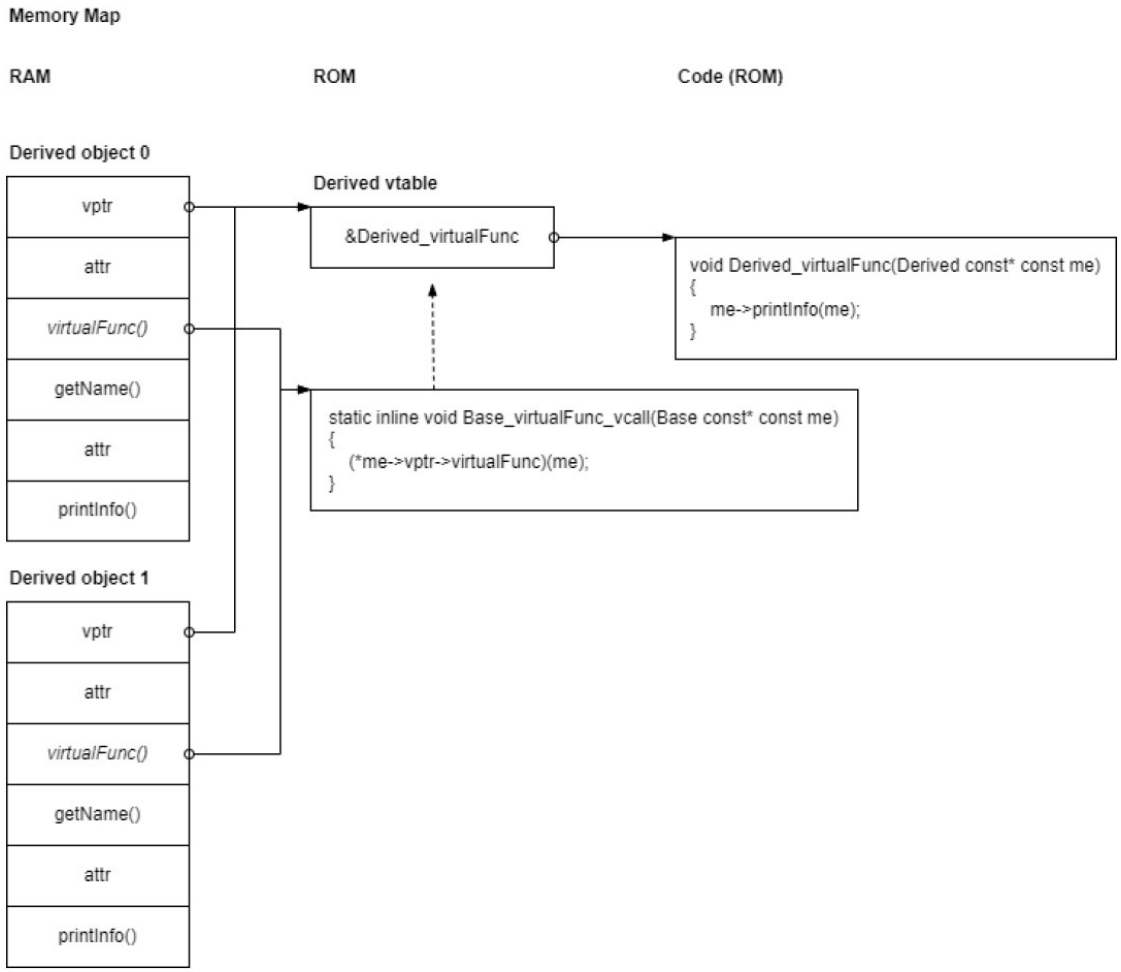
struct Derived
{
    Base super; // "Inheritance"
    DerivedAttr attr;
    void (*printInfo)(Derived const* const me);
}
```



The alignment of the *Derived* structure and the inherited members from the *Base* structure is guaranteed by the C Standard WG14/N1124.
Section 6.7.2.1.13 of this Standard, says: *"... A pointer to a structure object, suitably converted, points to its initial member. There may be unnamed padding within a structure object, but not at its beginning".*

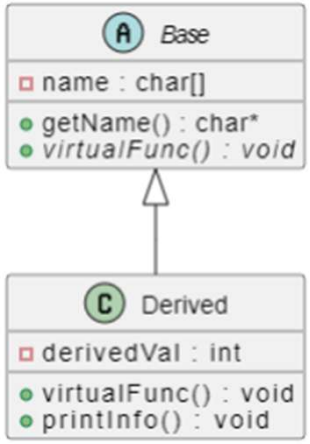
Polymorphism in C

- Virtual function table functions:
  - Base class: default impl. (or assertion for pure virt.)
  - Derived class: specific impl.
- Interface function pointers for late binding -> **Inline Virtual Call Forwarder functions**, such as Base_virtualFunc_vcall()



Polymorphism in C – Base - Example

- Header File base.h




```c
typedef struct Base Base;
typedef struct BaseAttr BaseAttr;
typedef struct BaseVtable BaseVtable;

struct BaseAttr
{
    char* name;
}

struct Base
{
    BaseVTable const* vptr;
    BaseAttr attr;
    void (*virtualFunc)(Base const* const me);      // void virtualFunc(): I/F, will be impl. by Derived
    const char* (*getName)(Base const* const me);   // const char* getName()
} ;

struct BaseVtable
{
    void (*virtualFunc)(Base const* const me);
};

// inline virtual Call:
static inline void Base_virtualFunc_vcall(Base const* const me)
{
    (*me->vptr->virtualFunc)(me);   // late binding: calls the func in the current virtual table
}
```

- Source File base.c

```c
static void Base_virtualFunc(Base const* const me);

void Base_init(Base* const me, const char* name)
{
    /* static const (immutable) virtual function table allocation, kept in ROM */
    static const BaseVtable vtable = {
        .virtualFunc = Base_virtualFunc         /* initialize pointer to local pure virtualFunc() */
    };
    me->vptr = &vtable;                         /* default assignment to own virtual table */
    me->attr.name = malloc(strlen(name)+1);
    memcpy(me->attr.name, name, strlen(name));
    me->attr.name[strlen(name)] = '\0';

    me->virtualFunc = Base_virtualFunc_vcall;  /* assign I/F with pointer to inline virtual call */

    me->getName = Base_getName;
}

/* empty "implementation" of the pure virtual function virtFunc() */
void Base_virtualFunc(Base const* const me)
{
    (void)me; /* avoid compiler warnings regarding unused parameter */
    assert(0); /* pure virtual function, shall never be called */
}

const char* Base_getName(Base const* const me)
{
    return me->attr.name;
}
```

Polymorphism in C – Derived - Example

- Header file derived.h

```c
#include "base.h"     /* the base class interface */
struct DerivedAttr;
{
    int derivedVal;
} DerivedAttr;

typedef struct Derived
{
    Base super;       /* inherits Base, must be at first position */
    DerivedAttr attr;
    void (*printInfo)(Derived const* const me);
} Derived;

void Derived_virtualFunc(Derived const* const me);
void Derived_printInfo(Derived const* const me);
```

- Source file derived.c

```c
void Derived_init(Derived* const me, const char* name, int val)
{
    /* static const (immutable) virtual func. table allocation, kept in ROM */
    static const BaseVtable vtable = {
        /* init ptr to own virtualFunc() impl. with "me" upcast */
        .virtualFunc = (void (*)(Base const* const me))Derived_virtualFunc
    };
    Base_init(&me->super, name);

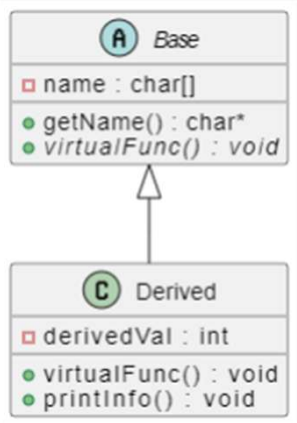
    /* assign own vtable to Base's vptr (overwrite) */
    me->super.vptr = &vtable;

    me->attr.derivedVal = val;

    me->printInfo = Derived_printInfo;
}

/* Derived's specific implementation of virtualFunc() */
void Derived_virtualFunc(Derived const* const me)
{
    me->printInfo(me);
}

void Derived_printInfo(Derived const* const me)
{
    printf("Name: %s - Val: %i\n", me->super.getName(&me->super), me->attr.derivedVal);
}
```

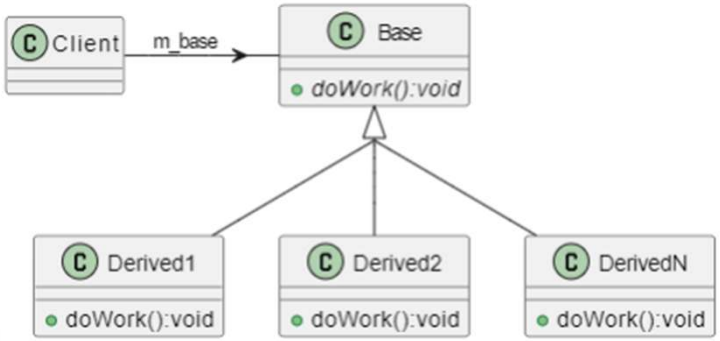


Naming conventions & idioms

- Naming
  - Prefixes: <Base>_, <Derived>_
  - Instance variable name of inherited class: **super**
  - Object pointer in parameter lists of member functions: **me**
  - Attribute instance variable: **attr**
  - Pointer to immutable virtual function table: **vptr**
  - Instance variable with immutable virtual function table: **vtable**

- Class Infrastructure Functions
  - Constructor: `ClassName* ClassName_create()`
  - Initializer: `ClassName_init(ClassName* const me)`
  - Destructor: `ClassName_destroy(ClassName* me)`
  - De-initializer: `ClassName_finalize(ClassName* me)`

Example: Polymorphic array




```
// create objects on Stack
Derived1 d1;
Derived2 d2;
Derived3 d3;

// initialize the objects
Derived1_init(&d1);
Derived2_init(&d2);
Derived3_init(&d3);

// use the objects
Base* b[3];
    b[0] = &d1;
    b[1] = &d2;
    b[2] = &d3;

for (int i = 0; i < 3; i++) {
    b[i]->doWork(b[i]);
}
```

```
Base* b[10];

// create objects on Heap
for (int i = 0; i < 10; i++) {
    b[i] = (Base*)Derived_create();
}

// use objects
for (int i = 0; i < 10; i++) {
    b[i]->doWork(b[i]);
}

// clean-up
for (int i = 0; i < 10; i++) {
    Derived_destroy(b[i]);
}
```

Conclusion

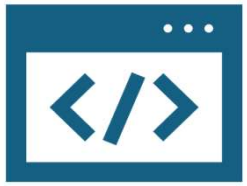


Key Points

OOP concepts possible in C

Requires discipline and following conventions

Always keep an eye on memory management



Further Reading

Witschi Confluence Knowledge base: Object Oriented Programming in C

Example library in GitHub: **lib.emb.oop-c-scheme**

Object-Oriented Programming in C by Dr. Miro Samek / Quantum Leaps

C Standard WG14/N1124

Questions?

